

```

"""
MLOps-style batch job: rolling mean signal generator on OHLCV data.
Usage:
    python run.py --input data.csv --config config.yaml --output metrics.json --l
"""

import argparse
import json
import logging
import sys
import time
from pathlib import Path

import numpy as np
import pandas as pd
import yaml

# -----
# Argument parsing
# -----

def parse_args():
    parser = argparse.ArgumentParser(description="MLOps batch signal job")
    parser.add_argument("--input", required=True, help="Path to input CSV file")
    parser.add_argument("--config", required=True, help="Path to YAML config fi")
    parser.add_argument("--output", required=True, help="Path to output metrics")
    parser.add_argument("--log-file", required=True, dest="log_file",
                        help="Path to output log file")
    return parser.parse_args()

# -----
# Logging setup
# -----

def setup_logging(log_file: str) -> logging.Logger:
    logger = logging.getLogger("mlops_job")
    logger.setLevel(logging.DEBUG)

    fmt = logging.Formatter(
        "%(asctime)s [%(levelname)s] %(message)s",
        datefmt="%Y-%m-%dT%H:%M:%S"
    )

```

```

# File handler
fh = logging.FileHandler(log_file, mode="w")
fh.setLevel(logging.DEBUG)
fh.setFormatter(fmt)
logger.addHandler(fh)

# Console handler (INFO and above)
ch = logging.StreamHandler(sys.stdout)
ch.setLevel(logging.INFO)
ch.setFormatter(fmt)
logger.addHandler(ch)

return logger

# -----
# Config loading + validation
# -----

def load_config(config_path: str, logger: logging.Logger) -> dict:
    path = Path(config_path)
    if not path.exists():
        raise FileNotFoundError(f"Config file not found: {config_path}")

    with open(path, "r") as f:
        cfg = yaml.safe_load(f)

    if not isinstance(cfg, dict):
        raise ValueError("Config YAML must be a mapping (key: value) document")

    required_keys = {"seed", "window", "version"}
    missing = required_keys - cfg.keys()
    if missing:
        raise ValueError(f"Config is missing required fields: {missing}")

    if not isinstance(cfg["seed"], int):
        raise ValueError(f"Config 'seed' must be an integer, got: {type(cfg['seed'])}")
    if not isinstance(cfg["window"], int) or cfg["window"] < 1:
        raise ValueError(f"Config 'window' must be a positive integer, got: {cfg['window']}")
    if not isinstance(cfg["version"], str):
        raise ValueError(f"Config 'version' must be a string, got: {type(cfg['version'])}")

    logger.info(
        f"Config loaded and validated - seed={cfg['seed']}, "
        f"window={cfg['window']}, version={cfg['version']}"
    )

```

```

    return cfg

# -----
# Dataset loading + validation
# -----

def load_dataset(input_path: str, logger: logging.Logger) -> pd.DataFrame:
    path = Path(input_path)
    if not path.exists():
        raise FileNotFoundError(f"Input file not found: {input_path}")

    try:
        df = pd.read_csv(path)
    except Exception as exc:
        raise ValueError(f"Failed to parse CSV: {exc}") from exc

    if df.empty:
        raise ValueError("Input CSV is empty (zero rows)")

    if "close" not in df.columns:
        raise ValueError(
            f"Required column 'close' not found. Columns present: {list(df.columns)}"
        )

    # Coerce close to numeric; rows that can't be converted become NaN
    df["close"] = pd.to_numeric(df["close"], errors="coerce")
    invalid_rows = df["close"].isna().sum()
    if invalid_rows > 0:
        logger.warning(f"{invalid_rows} rows had non-numeric 'close' values and w
        df = df.dropna(subset=["close"]).reset_index(drop=True)

    if df.empty:
        raise ValueError("No valid numeric 'close' values remain after cleaning")

    logger.info(f"Dataset loaded: {len(df)} rows, columns: {list(df.columns)}")
    return df

# -----
# Core processing
# -----

def compute_rolling_mean(df: pd.DataFrame, window: int, logger: logging.Logger) -
    """
    Compute rolling mean on 'close' with the given window.
    The first (window-1) rows will have NaN for rolling_mean;

```

```

those rows are excluded from signal computation (signal = NaN there).
"""
df = df.copy()
df["rolling_mean"] = df["close"].rolling(window=window).mean()
logger.info(
    f"Rolling mean computed (window={window}); "
    f"first {window - 1} row(s) have NaN rolling_mean and are excluded from s
)
return df

def compute_signal(df: pd.DataFrame, logger: logging.Logger) -> pd.DataFrame:
    """
    signal = 1 if close > rolling_mean, else 0.
    Rows where rolling_mean is NaN are excluded (signal left as NaN).
    """
    df = df.copy()
    mask = df["rolling_mean"].notna()
    df.loc[mask, "signal"] = (df.loc[mask, "close"] > df.loc[mask, "rolling_mean"])
    valid_signals = int(mask.sum())
    logger.info(
        f"Signal generated for {valid_signals} rows "
        f"({len(df) - valid_signals} rows excluded due to NaN rolling_mean)"
    )
    return df

# -----
# Metrics writing
# -----

def write_metrics(output_path: str, payload: dict, logger: logging.Logger) -> Non
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w") as f:
        json.dump(payload, f, indent=2)
    logger.info(f"Metrics written to {output_path}")

# -----
# Main
# -----

def main():
    args = parse_args()
    logger = setup_logging(args.log_file)

```

```

logger.info("=" * 60)
logger.info("Job started")
logger.info(f"  input:      {args.input}")
logger.info(f"  config:     {args.config}")
logger.info(f"  output:     {args.output}")
logger.info(f"  log-file:   {args.log_file}")
logger.info("=" * 60)

t_start = time.time()

# Default version for error output (may be overridden once config is loaded)
version = "unknown"

try:
    # --- 1. Load + validate config ---
    cfg = load_config(args.config, logger)
    version = cfg["version"]
    seed    = cfg["seed"]
    window  = cfg["window"]

    # Set random seed for reproducibility
    np.random.seed(seed)
    logger.info(f"Random seed set: {seed}")

    # --- 2. Load + validate dataset ---
    df = load_dataset(args.input, logger)

    # --- 3. Rolling mean ---
    df = compute_rolling_mean(df, window, logger)

    # --- 4. Signal generation ---
    df = compute_signal(df, logger)

    # --- 5. Metrics ---
    valid_signal_mask = df["signal"].notna()
    rows_processed    = int(valid_signal_mask.sum())
    signal_rate       = round(float(df.loc[valid_signal_mask, "signal"].mean())
    latency_ms        = round((time.time() - t_start) * 1000)

    logger.info(f"rows_processed = {rows_processed}")
    logger.info(f"signal_rate    = {signal_rate}")
    logger.info(f"latency_ms     = {latency_ms}")

    metrics = {
        "version":      version,
        "rows_processed": rows_processed,
        "metric":       "signal_rate",

```

```

        "value":          signal_rate,
        "latency_ms":     latency_ms,
        "seed":           seed,
        "status":         "success",
    }

    write_metrics(args.output, metrics, logger)

    logger.info("Job completed successfully")
    logger.info("=" * 60)

    # Print final JSON to stdout (required by Docker spec)
    print(json.dumps(metrics, indent=2))
    sys.exit(0)

except Exception as exc: # noqa: BLE001
    logger.error(f"Job failed: {exc}", exc_info=True)

    error_metrics = {
        "version":        version,
        "status":         "error",
        "error_message":  str(exc),
    }

    try:
        write_metrics(args.output, error_metrics, logger)
    except Exception as write_exc:
        logger.error(f"Could not write error metrics: {write_exc}")

    logger.info("Job ended with error")
    logger.info("=" * 60)

    print(json.dumps(error_metrics, indent=2))
    sys.exit(1)

if __name__ == "__main__":
    main()

```